

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра САУ

ОТЧЕТ
по лабораторной работе № 1
«Работа с механическим энкодером»
по дисциплине «Микропроцессорная техника в мехатронике и
робототехнике»

Студент гр. 2491

Недовизий А.И.

Преподаватель

Копычев М.М.

Санкт-Петербург

2025

Цель работы:

Изучить принцип работы механического энкодера и научиться работать с внешними прерываниями.

Задание на работу №1:

Написать программу вывода значения поворота энкодера на семисегментные индикаторы, используя внешнее прерывание INT1 для определения направления вращения энкодера (на индикаторы должны выводиться как положительные, так и отрицательные значения от -100 до 100). По нажатию на кнопку энкодера, подключенную к INT2, счётчик оборотов должен обнулиться.

Основные сведения:

Для того чтобы функция обработчик прерывания была вызвана необходимо выполнение трёх условий:

- 1) Прерывания должны быть разрешены глобально, делается это установкой в «1» бита I регистра SREG. Для этого выделена специальная машинная команда контроллера – sei. Для сброса этого бита также выделена отдельная команда – cli. Для использования ассемблерных команд в VSCODE используется функция sei(). Таким образом, для глобального разрешения прерываний необходимо добавить строчку sei() в одну из функций программы, например в функцию main.
- 2) Должно быть разрешено соответствующее прерывание, делается это с помощью конфигурации соответствующих регистров.
- 3) Должно произойти событие вызывающее соответствующее прерывание, например, переполнение таймера или изменение сигнала на входе внешнего прерывания

Запрос прерывания – это событие, которое происходит, если внешний или внутренний источник прерывания разрешен локально (в случае внешнего прерывания – редактируется EIMSK) и правильно настроен (в случае внешнего прерывания – редактируются EICRA/EICRB).

Механический энкодер предназначен для ввода информации (например, для плавного изменения параметров), как более надежная и удобная замена потенциометру. В отличие от потенциометра энкодер не имеет ограничения угла поворота.

Принцип работы энкодера представлен на рисунке 1. При вращении энкодера на его выходах А и В (PD0 (INT0) и PD1 (INT1) контроллера, соответственно) формируются импульсы. Если вращать энкодер вправо, то импульс А будет незначительно опережать импульс В, если вращать влево, то - наоборот.

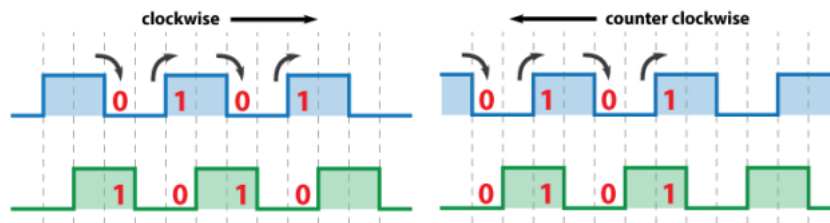


Рисунок 1- Принцип работы механического энкодера

Таким образом, считая импульсы на любом из выходов можно определить на какой угол повернули энкодер, а по последовательности выходов можно определить направление вращения.

Схема подключения энкодера приведена на рисунке 2.

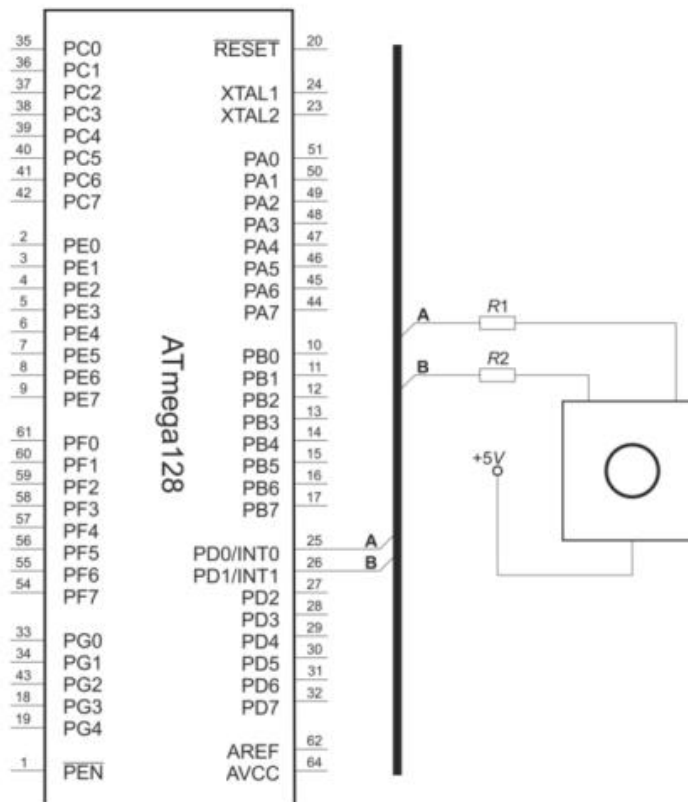


Рисунок 2 – Схема подключения энкодера

Подключение семисегментных индикаторов приведено на рисунке 3.

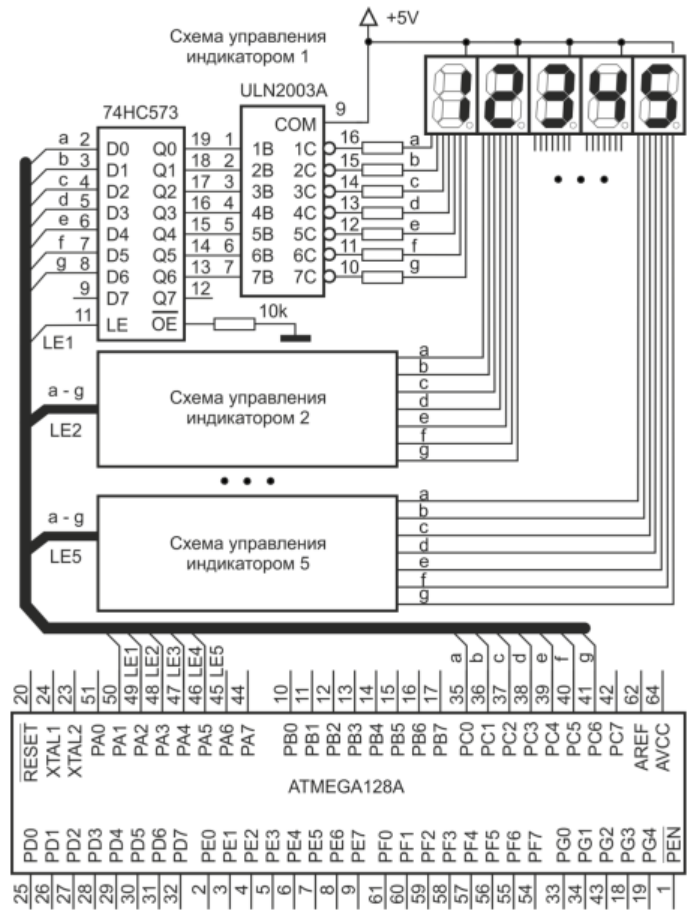


Рисунок 3 - Подключение семисегментных индикаторов

Код работы:

Код программы представлен на рисунках 4-8 (Листинг).

```
1  #include <avr/io.h>
2  #include <util/delay.h>
3  #include <avr/interrupt.h>
4  #include <math.h>
5
6  #define EMPTY 10
7  #define MINUS 11
8
9  void clearSegments(void);
10 void numout(int32_t number);
11 void drawposneg(int32_t number);
12 int8_t Digit(int32_t d, uint8_t m);
13
14 int32_t angle = 0;
15 const uint8_t segments[] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F, 0, 0x40};
16 // добавить функцию digit и нормально выводить число angle +добавить ограничения и минусы
17
18 int main(void)
19 {
20     DDRC = 0xFF;
21     DDRA = (1 << 1) | (1 << 2) | (1 << 3) | (1 << 4) | (1 << 5);
22     clearSegments();
23     EIMSK = (1 << 1) | (1 << 2);
24     EICRA = (1 << ISC11) | (0 << ISC20) | (1 << ISC21);
25     sei();
26     while (1)
27     {
28         numout(angle);
29     }
30 }
```

Рисунок 4 – Код программы

```

31
32 int8_t Digit(int32_t d, uint8_t m)
33 {
34
35     int8_t i = 5, a;
36     if (d < 0)
37     |   d *= -1;
38     while (i)
39     {
40         // цикл по разрядам числа
41         a = d % 10; // выделяем очередной разряд
42         if (i-- == m)
43         |   break; // выделен заданный разряд - уходим
44         d /= 10; // уменьшаем число в 10 раз
45     }
46     return (a);
47 }
48 void numout(int32_t number) // функция, которая выводит цифру
49 {
50     if ((number > 9999)|(number < -9999))
51     |   return;
52
53     if (number == 0)
54     {
55         clearSegments();
56         PORTA |= (1 << 5); // подача в регистр нужной последовательности, чтобы выбрать нужный разряд
57         PORTC = segments[0]; // подача изображения нужной цифры
58         _delay_us(10);
59         PORTA &= ~(1 << 5);
60         return;

```

Рисунок 5 – Код программы (продолжение)

```

61     }
62
63     uint8_t i = 2; // номер разряда
64
65     while (Digit(number, i) == 0) // с помощью этого цикла мы убираем нули
66     {
67         PORTA |= (1 << i);
68         PORTC = 0x0;
69         _delay_us(20); // пауза
70         PORTA &= ~(1 << i);
71         i++; // переход в следующий разряд
72     }
73
74     while (i <= 5) // запоминает i при котором у нас первая не нулевая цифра и выводит последовательно оставшиеся цифры в разряды
75     {
76         PORTC = segments[Digit(number, i)];
77         PORTA |= (1 << i);
78         // подача в регистр нужной последовательности, чтобы выбрать нужный разряд
79         _delay_us(20); // подача в регистр нужной последовательности, чтобы выбрать нужный разряд
80         PORTA &= ~(1 << i); // ставим регистр в изначальное значение
81         i++; // переход в следующий разряд
82     }
83     drawposneg(number);
84 }
85
86 void drawposneg(int32_t number)
87 {
88     if (number < 0)

```

Рисунок 6 – Код программы (продолжение)

```

89     {
90         PORTA |= (1 << 1);          // подача в регистр нужной последовательности, чтобы выбрать нужный разряд
91         PORTC = segments[MINUS]; // подача изображения нужной цифры
92         _delay_us(10);
93         PORTA &= ~(1 << 1);
94     }
95     else
96     {
97         PORTA |= (1 << 1); // подача в регистр нужной последовательности, чтобы выбрать нужный разряд
98         PORTC = segments[EMPTY];
99         _delay_us(10);
100        PORTA &= ~(1 << 1);
101    }
102 }
103
104 void clearSegments(void)
105 {
106     for (uint8_t i = 1; i <= 5; i++)
107     {
108         PORTA |= (1 << i);
109         PORTC = 0;
110         _delay_us(10);
111         PORTA &= ~(1 << i);
112     }
113 }
114
115 ISR(INT1_vect)
116 {
117     if ((PIND & (1 << 0)) != 0)

```

Рисунок 7 – Код программы (продолжение)

```

118     {
119         EICRA |= (1 << ISC11);
120         if ((PIND & (1 << 1)) != 0)
121             angle--;
122         else
123             angle++;
124     }
125     else
126     {
127         EICRA |= (1 << ISC11) | (1 << ISC10);
128         if ((PIND & (1 << 1)) != 0)
129             angle++;
130         else
131             angle--;
132     }
133
134     if (angle > 9999)
135         angle = 9999;
136     else if (angle < -9999)
137         angle = -9999;
138 }
139
140
141 ISR(INT2_vect)
142 {
143     angle = 0;
144 }

```

Рисунок 8 – Код программы (продолжение)

Блок-схемы программы:

а) Основное тело программы (int main(void))

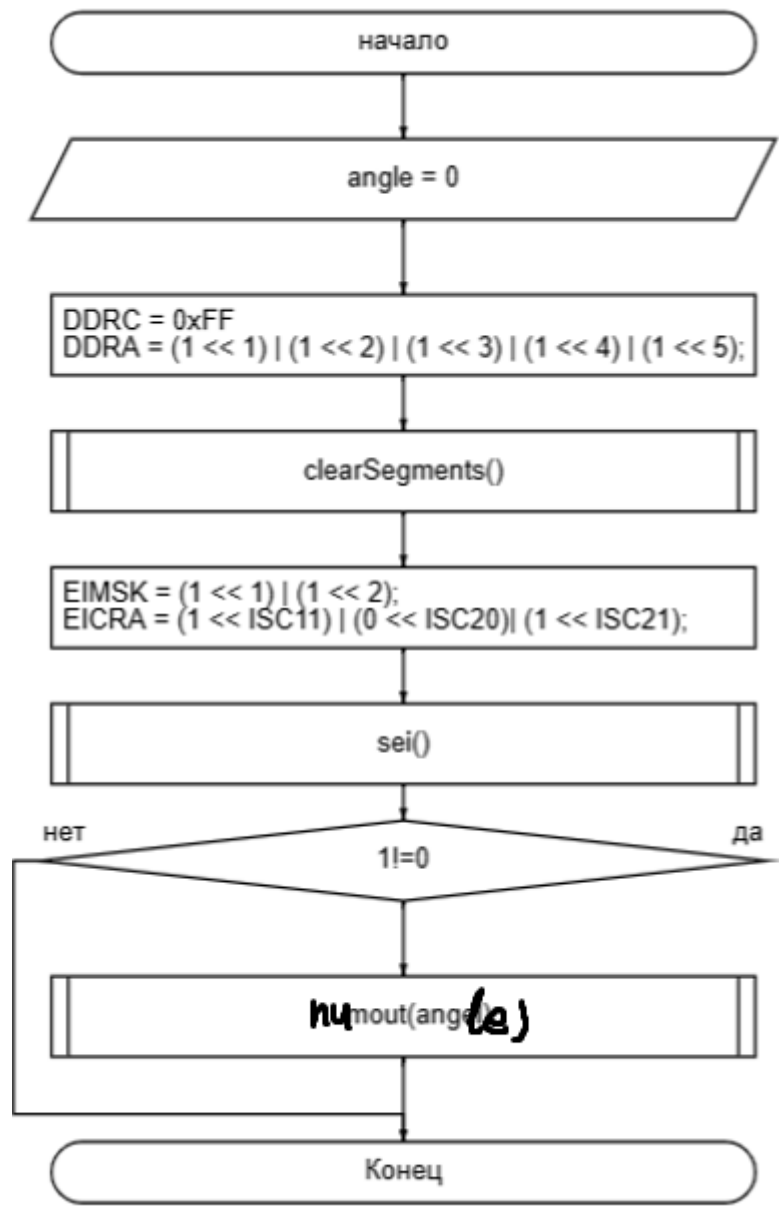


Рисунок 9 – Блок-схема функции `int main(void)`

б) Функция `uint8_t Digit (int16_t d, uint8_t m)`

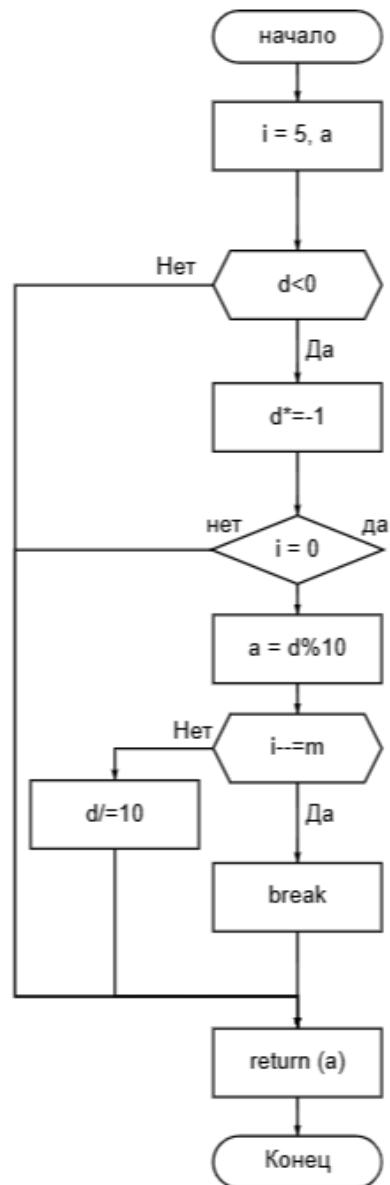


Рисунок 10 – Блок-схема функции `uint8_t Digit(int16_t d, uint8_t m)`

в) Функция `ISR(INT2_vect)`

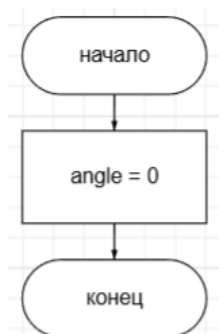


Рисунок 11 – Блок-схема функции `ISR(INT2_vect)`

г) Функция ISR(INT0_vect)

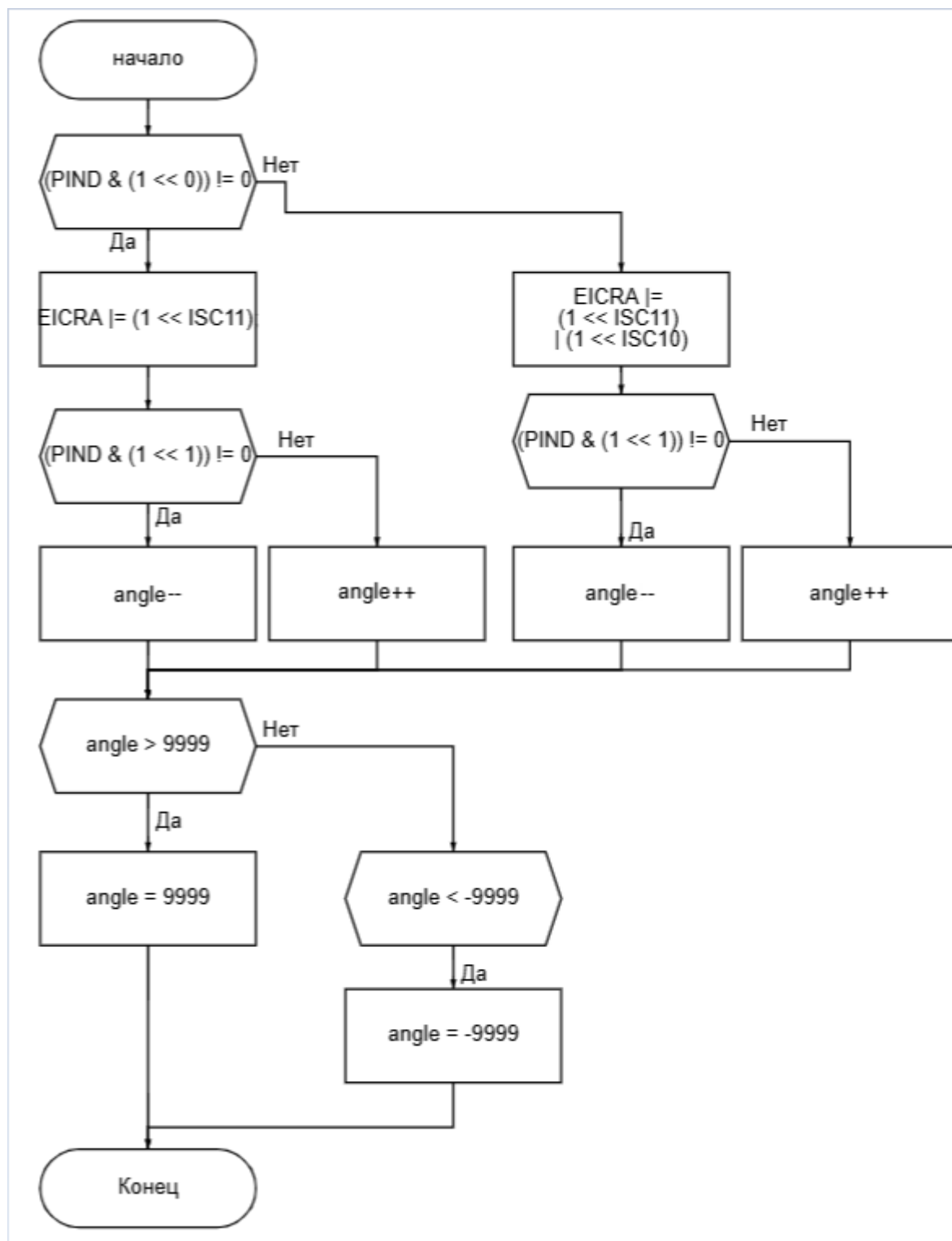


Рисунок 12 – Блок-схема функции ISR(INT0_vect)

д) Функция void numout (int32_t number)

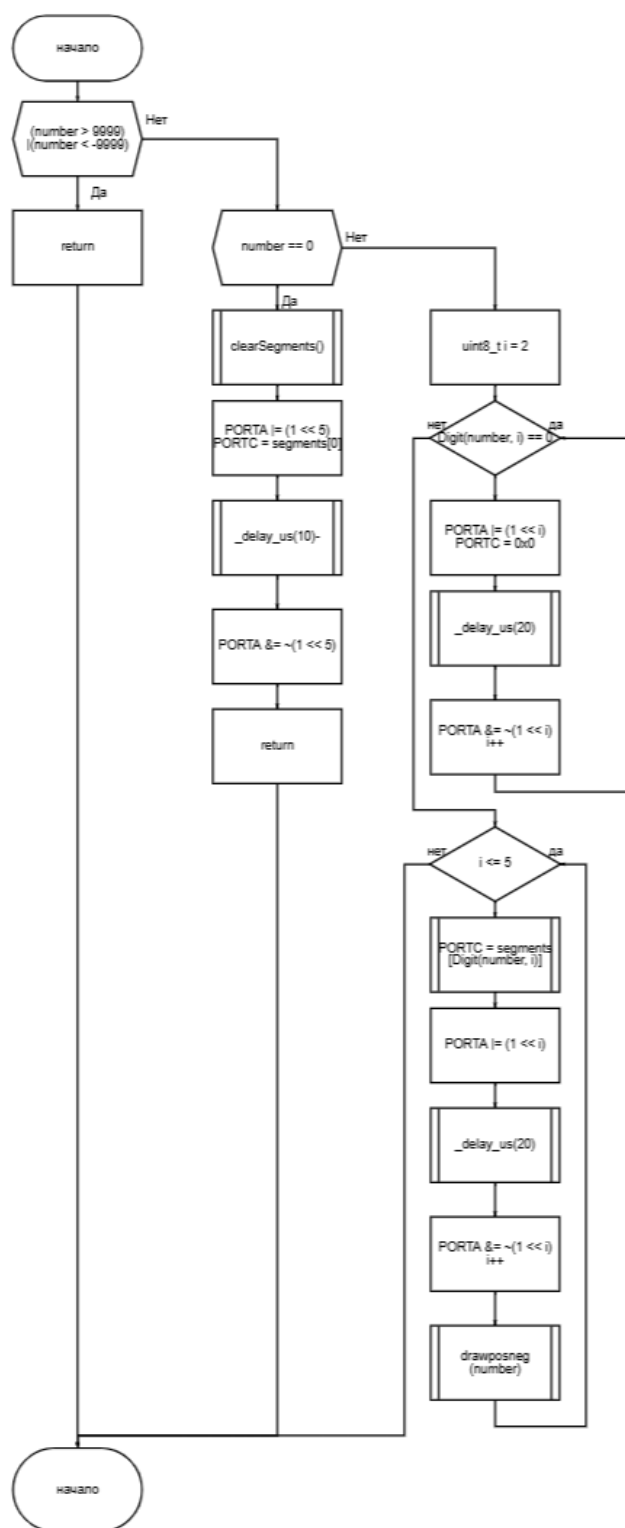


Рисунок 13 – Блок-схема функции void numout (int32_t number)

д) Функция `void drawposneg(int32_t number)`

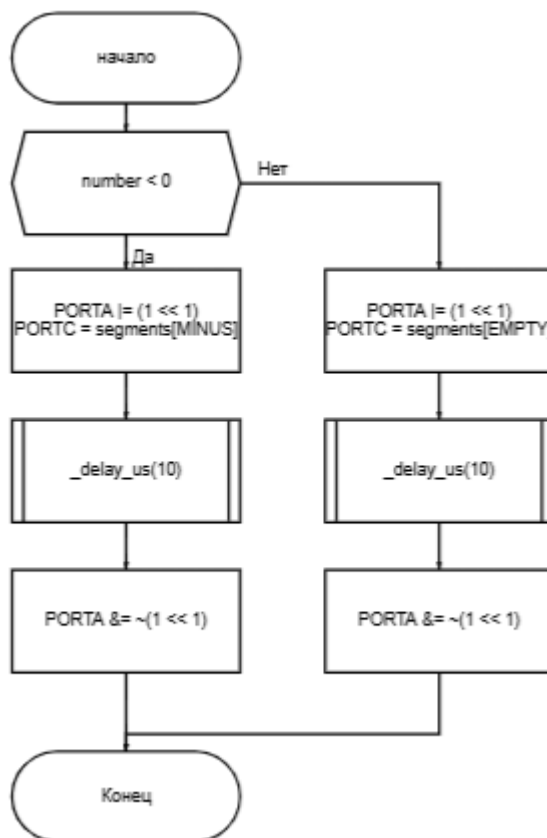


Рисунок 14 – Блок-схема функции `void drawposneg(int32_t number)`

д) Функция void clearSegments(void)

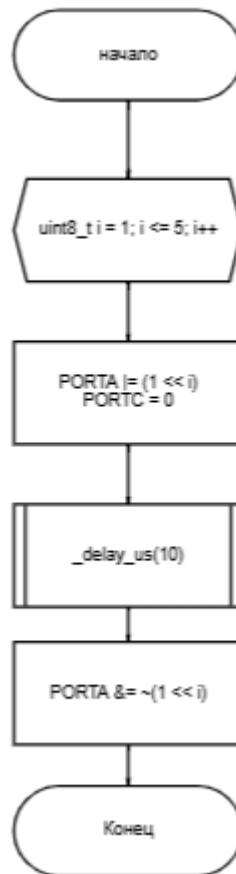


Рисунок 15 – Блок-схема функции void clearSegments(void)

Вывод:

В ходе выполнения лабораторной работы были закреплены практические навыки работы с семисегментными индикаторами и изучен принцип считывания данных с энкодера с применением прерываний. Была разработана программа, которая отслеживает вращение энкодера и отображает на индикаторах количество его переключений. Алгоритм программы различает направление вращения: при повороте по часовой стрелке значение счетчика увеличивается на единицу (до 100), а против часовой стрелки — уменьшается (до -100). При нажатии на кнопку счетчик сбрасывается до нуля.